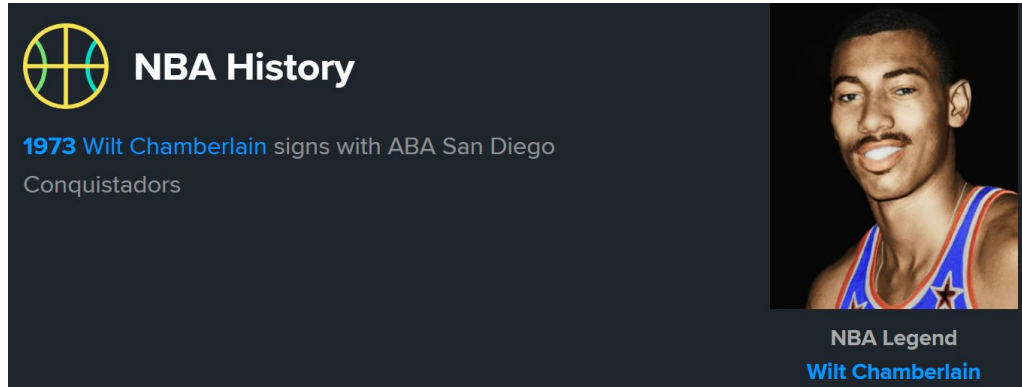


# COMS W4156 Advanced Software Engineering (ASE)

September 26, 2024

On this day ...

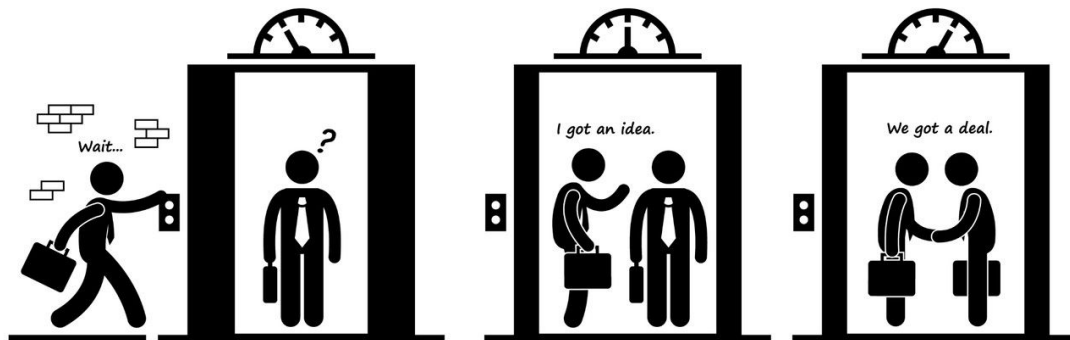


# Agenda

1. Team Proposal Pitches
2. if time permits, code review
3. Upcoming



## ELEVATOR PITCH



# Let's Hear Your Pitch

Assuming the pre-defined Breakout Rooms work,  
or after I otherwise get them set up,  
please go to your team's breakout room

One of the Mentors (not necessarily your team's Mentor)  
will eventually join your team's Breakout Room, please wait

When a Mentor arrives, please briefly pitch your team project idea

The Mentor will provide quick feedback and then move on to another team

Your team should then discuss further until I send notice that Breakout Rooms are closing, when you should return to main room for the rest of class



# Agenda

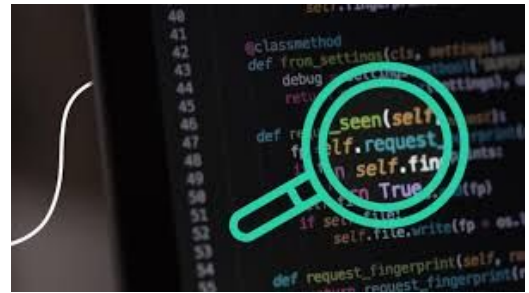
1. Team Proposal Pitches
2. if time permits, code review
3. Upcoming



# Motivation for Code Review

In “[code review](#)”, one or more *human developers* examine the code without executing it, usually only a small part of the codebase (e.g., [pull request](#))

Like automated static analysis, code review can potentially consider *all* inputs whereas testing and other dynamic analyses can only consider *some* inputs - often a very small subset of all inputs



Code review can find problems that static and dynamic tools cannot, such as code that passes all static and dynamic checks, but is still [unreadable](#). *Might* also detect flaws beyond the state of the art of testing and analysis (e.g., [zero-day vulnerabilities](#))

# Example Unreadable Code - What Does This Code Do?

The reviewer does not need to understand what this code does, only that it is unreadable, to send back to the developer to fix



```
#include <stdio.h>

#define N(a)      "%"#a"$hhn"
#define O(a,b)    "%10$"#a"d"N(b)
#define U         "%10$.*37$d"
#define G(a)      "%"#a"$s"
#define H(a,b)    G(a)G(b)
#define T(a)      a a
#define s(a)      T(a)T(a)
#define A(a)      s(a)T(a)a
#define n(a)      A(a)a
#define D(a)      n(a)A(a)
#define C(a)      D(a)a
#define R         C(C(N(12)G(12)))
#define o(a,b,c)  C(H(a,a))D(G(a))C(H(b,b)G(b))n(G(b))O(32,c)R
#define SS        O(78,55)R "\n\033[2J\n*26$s";
#define E(a,b,c,d) H(a,b)G(c)O(253,11)R G(11)O(255,11)R H(11,d)N(d)O(253,35)R
#define S(a,b)    O(254,11)H(a,b)N(68)R G(68)O(255,68)N(12)H(12,68)G(67)N(67)

char* fmt = O(10,39)N(40)N(41)N(42)N(43)N(66)N(69)N(24)O(22,65)O(5,70)O(8,44)N(
45)N(46)N(47)N(48)N(49)N(50)N(51)N(52)N(53)O(28,
54)O(5,55)O(2,56)O(3,57)O(4,58)O(13,73)O(4,
71)N(72)O(20,59)N(60)N(61)N(62)N(63)N(64)R R
E(1,2,3,13)E(4,5,6,13)E(7,8,9,13)E(1,4,7,13)E
(2,5,8,13)E(3,6,9,13)E(1,5,9,13)E(3,5,7,13
)E(14,15,16,23)E(17,18,19,23)E(20,21,22,23)E
(14,17,20,23)E(15,18,21,23)E(16,19,22,23)E(16,18,20,23)R U O(255,38)R G(38)O(255,36)
R H(13,23)O(255,11)R H(11,36)O(254,36)R G(36)O(
255,36)R S(1,14)S(2,15)S(3,16)S(4,17)S(5,18)S(6,
19)S(7,20)S(8,21)S(9,22)H(13,23)H(36,67)N(11)R
G(11)""O(255,25)R s(C(G(11)))n(G(11))G(
11)N(54)R C("aa")s(A(G(25)))T(G(25))N
(14,1,26)O(15,2,27)O(16,3,28)O(17,4,29)O(18
,5,30)O(19,6,31)O(20,7,32)O(21,8,33)O(22,9,
34)n(C(U))N(68)R H(36,13)G(23)N(11)R C(D(G(11)))
D(G(11))G(68)N(68)R G(68)O(49,35)R H(13,23)G(67)N(11)R C(H(11,11)G(
11))A(G(11))C(H(36,36)G(36))s(G(36))O(32,58)R C(D(G(36)))A(G(36))JS

#define arg d+6,d+8,d+10,d+12,d+14,d+16,d+18,d+20,d+22,0,d+46,d+52,d+48,d+24,d\
+26,d+28,d+30,d+32,d+34,d+36,d+38,d+40,d+50,(scanf(d+126,d+4),d+(6\
-2)+18*(1-d[2]%2+d[4]*2),d,d+66,d+68,d+70,d+78,d+80,d+82,d+90,d+\
92,d+94,d+97,d+54,d[2],d+2,d+71,d+77,d+83,d+89,d+95,d+72,d+73,d+74\
,d+75,d+76,d+84,d+85,d+86,d+87,d+88,d+100,d+101,d+96,d+102,d+99,d+\
67,d+69,d+79,d+81,d+91,d+93,d+98,d+103,d+58,d+60,d+98,d+126,d+127,\
d+128,d+129

char d[538] = {1,0,10,0,10};

int main() {
    while(*d) printf(fmt, arg);
}
```

# Lightweight Code Review



Many organizations require code to be reviewed by a peer, a senior developer, or a designated reader prior to committing to a shared branch in the code repository

*Lightweight* forms of code review involve one or two readers besides the coder

May be synchronous (reviewer sits with coder, typically immediately after coder “done”), or asynchronous (reviewer goes through the code on their own)

Asynchronous generally considered better - developer submits prospective changes for review and works on something else until response

# Training



Although there are [major problems with synchronous reviews](#) (such as context switching and time pressure), synchronous code review by senior developer is valuable for on-boarding and training junior developers

What does the code reviewer do?  
See [code review best practices](#)

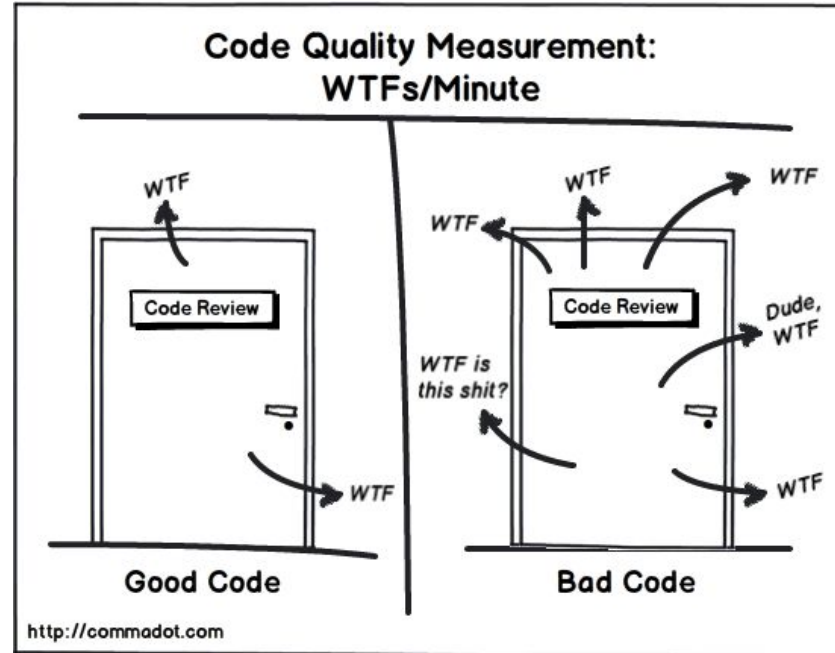
# Formal Code Review

*Formal* code review (sometimes called [Fagan inspection](#)) is heavyweight - involves an elaborate *team* review process

Goal is to identify as many defects as possible, including logic errors, missing requirements, and documentation inconsistencies

Mandated for [safety-critical software](#)

Rarely used for conventional business and consumer software other than for training purposes



# Fagan Inspection Roles and Process



**Author:** The developer who wrote the code being inspected.

**Reader:** A team member (not the author) who presents the code to the group, reading it aloud line by line during the Inspection.

**Moderator:** Facilitates the discussion, keeps the inspection on track, and manages the identified defects. Often from a different team or a dedicated QA group.

**Inspectors:** All the other team members, who actively look for potential issues and raise concerns (one of them takes notes).

**Preparation before meeting** - code distributed to participants, who independently read through the code

**Inspection meeting** - walkthrough the code under inspection (possibly diverging through dependencies), discuss any defects found, rate severity (low, medium, high, severe)

**Followup actions** - The author addresses the defects, documents corrective actions taken, Moderator signs off. If particularly severe defects had been found, possibly another Inspection meeting

# Agenda

1. Team Proposal Pitches
2. if time permits, code review
3. **Upcoming**



coming, forthcoming,  
approaching, imminent,  
impending, nearing, looming,  
future, oncoming, next



 Thesaurus.plus

# Upcoming Deadlines

[Part Individual Miniproject part 3](#) due **tomorrow** September 27

[Team Project Proposal](#) due **tomorrow** September 27

[Team First Iteration](#) due Friday October 18 (3 weeks to build version 1)

[Team First Iteration Demo](#) due October 25  
(demo for your mentor sometime during  
the week following the first iteration deadline,  
your mentor must be able to "drive")



# Next Week

Finish anything we didn't get to today

RPC-based APIs

task boards and issue trackers





# Ask Me Anything